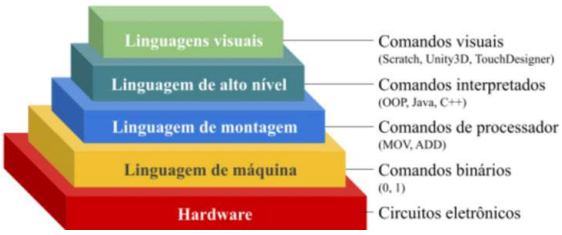
Universidade Estadual do Piauí – UESPI
Campus de Parnaíba
Tecnologia em Sistemas de Computação

Introdução a Computação

Aula 14 – Linguagem de Programação de Baixo Nível e Pseudocódigo



Níveis de linguagens de programação. (Carlos de Oliveira, 2020).

Prof. Francisco Rocha

LP de Baixo Nível e Pseudocódigo

- **Objetivos**

– Listar as operações que um computador pode realizar.
– Descrever as características importantes da máquina virtual Pep/8.
– Distinguir entre modo de endereçamento imediato e modo de endereçamento direto.
– Escrever um programa simples em linguagem de máquina.
– Distinguir entre linguagem de máquina e linguagem assembly.
– Descrever os passos para criar e executar um programa em linguagem assembly.
– Escrever um programa simples em linguagem assembly.
– Distinguir entre instruções para o montador e instruções a serem traduzidas.
– Distinguir entre seguir um algoritmo e desenvolver um.
– Descrever as construções de pseudocódigo usadas para expressar um algoritmo.
– Usar pseudocódigo para expressar um algoritmo.
– Descrever duas abordagens para testes.
– Projetar e implementar um plano de testes para um programa simples em linguagem assembly.

LP de Baixo Nível e Pseudocódigo

- **Computador**

– Um dispositivo eletrônico programável que pode armazenar, recuperar e processar dados.

- **Palavras importantes**: programável, armazenar, recuperar e processar.

– Os **dados** e as **instruções para manipular** os dados são logicamente iguais e podem ser armazenados no mesmo local.

– Que operações um computador pode executar?

LP de Baixo Nível e Pseudocódigo

- **Linguagem de Máquina**

– Linguagem composta por instruções codificadas em binário, incorporadas ao hardware de um computador específico e usadas diretamente por ele.

– Antes, por que alguém usaria linguagem de máquina?

- Dica: não tinham escolha. Por quê?

– **Características de uma linguagem de máquina**:

- Cada tipo de processador possui seu próprio conjunto de instruções de máquina específicas.
- A relação entre o processador e as instruções que ele pode executar é completamente integrada.
- Cada instrução de linguagem de máquina executa apenas uma tarefa de nível muito baixo.

LP de Baixo Nível e Pseudocódigo

- **O Computador Virtual Pep/8**

– **Computador virtual** é uma máquina hipotética projetada para conter as características importantes de um computador real que desejamos ilustrar.

– **Pep/8** é um computador virtual projetado por Stanley Warford que possui 39 instruções em linguagem de máquina.

- Não, não vamos abordar todas elas!

LP de Baixo Nível e Pseudocódigo

- **Pep/8: recursos**

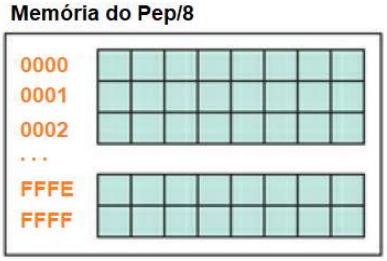
– **Recursos do Pep/8**

- Sua unidade de memória é composta por 65.536 bytes de armazenamento, numerados de 0 a 65.535 (em decimal).
- O comprimento da palavra é de 2 bytes (ou 16 bits).
- Possui sete registradores. A seguir três deles:
  - O contador de programa (PC, *Program Counter*) contém o endereço da próxima instrução a ser executada.
  - O registrador de instruções (IR, *Instruction Register*) contém uma cópia da instrução que está sendo executada.
  - O acumulador (AC) o registrador A, usado para armazenar dados e os resultados de operações.

LP de Baixo Nível e Pseudocódigo

- **Pep/8: diagrama da CPU e da memória**





– Os endereços estão em laranja, expressos em hexadecimal.
  - Não são armazenados na RAM.
  - Servem para indexar os bytes.
– Os bytes serão referenciados sempre pelo seu endereço.

7

LP de Baixo Nível e Pseudocódigo

- **Pep/8: formato da instrução**

– No processo de FDX, após ser buscada, a instrução entra no IR, é decodificada e então executada.

– Uma instrução do Pep/8 tem duas partes:
  - o especificador de instrução e
  - (opcionalmente) o especificador de operando de 16 bits.

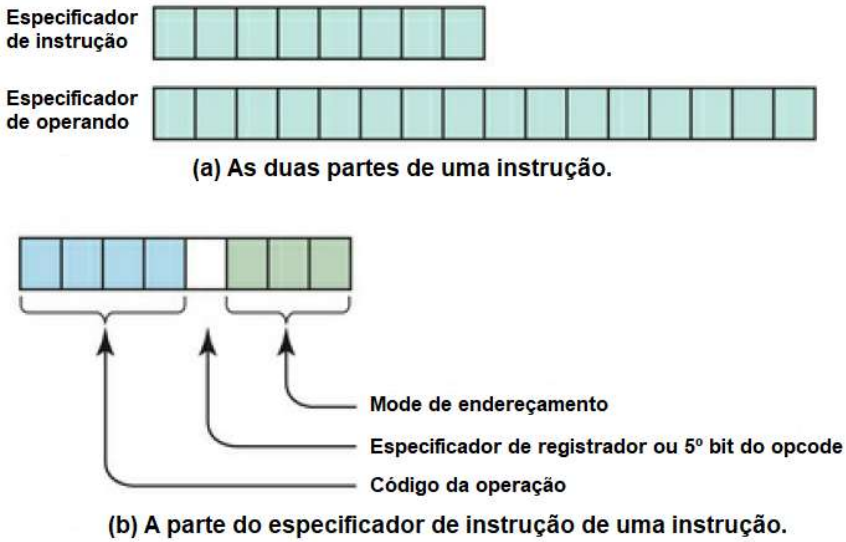
– **Especificador de instrução**: especifica qual instrução deve ser executada, e.g., adicionar um número a um valor já armazenado em um registrador, e como interpretar exatamente onde o operando está.

– **Especificador de registrador**: contém o próprio operando ou o endereço de onde o operando deve ser encontrado. Algumas instruções não usam o especificador de operando.

8

LP de Baixo Nível e Pseudocódigo

- **Pep/8: formato da instrução**



9

LP de Baixo Nível e Pseudocódigo

- **Pep/8: formato da instrução**

– No Pep/8, os códigos de operação (chamados **opcodes**) variam de 4 a 8 bits de comprimento. Abordamos opcodes de 4 ou 5 bits, com o quinto bit dos opcodes de 4 bits usado para especificar qual registrador usar.

– O especificador de registrador é bit 0 para o registrador A (acumulador), que é o único registrador que usaremos.

– O especificador de modo de endereçamento (3 bits a esquerda), indica como interpretar a parte do operando da instrução.

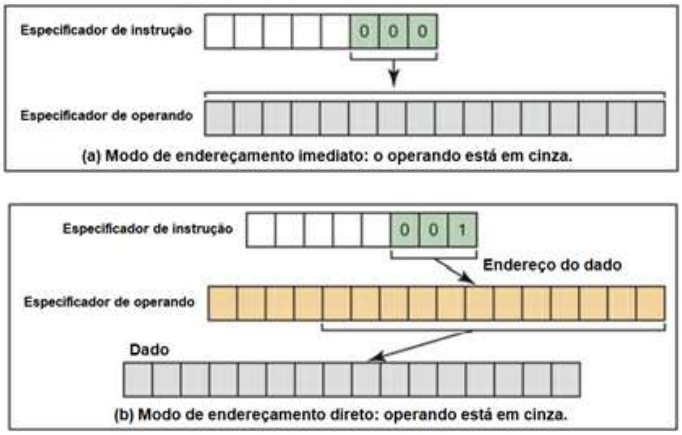
- Se **000**, o operando está no especificador de operando da instrução. Modo de endereçamento chamado de imediato (i).
- Se **001**, o operando é o endereço de memória nomeado no especificador de operando. Modo de endereçamento direto (d).

10

LP de Baixo Nível e Pseudocódigo

- **Pep/8: formato da instrução**

– Diferença entre o modo de endereçamento imediato e o modo de endereçamento direto.



11

11

LP de Baixo Nível e Pseudocódigo

- **Pep/8: exemplos de instruções**

– Algumas instruções do Pep/8

Opcode	Instrução
0000	Parar a execução.
1100	Carregar o operando dentro do registrador A.
1110	Carregar o conteúdo do registrador A dentro do operando.
0111	Adicionar o operando ao registrador A.
1000	Subtrair o operando do registrador A.
01001	Entrar caractere no operando.
01010	Saída de caractere do operando.

– **0000**: *Parar a execução*: Durante o ciclo FDX, quando o código da operação é todo zero, o programa é interrompido. Parar é uma instrução unária, portanto, ocupa apenas um byte. Os três bits mais à direita no byte são ignorados.

12

12

LP de Baixo Nível e Pseudocódigo

- **Pep/8: exemplos de opcodes de instruções**

– **1100**: *Carregar o operando no registrador A*: Carrega uma palavra (dois bytes) no registrador A. O especificador de modo determina onde a palavra está localizada.
  - **000** determina que o valor a ser carregado está na parte do operando da instrução (o segundo e o terceiro bytes da instrução); ou
  - **001** no local nomeado no operando.

– **1110**: *Armazena o registrador A no operando*: Esta instrução armazena o conteúdo do registrador A no local especificado no operando.

LP de Baixo Nível e Pseudocódigo

- **Pep/8: exemplos de opcodes de instruções**

– **0111**: *Adiciona o operando ao registrador A*: Assim como a operação de carregamento, a operação de adição usa o especificador do modo de endereçamento, oferecendo interpretações alternativas. As duas alternativas são explicadas nas figuras.

– **1000**: *Subtrai o operando*: Instrução semelhante à operação de adição, exceto que o operando é subtraído do registrador A. Assim como nas operações de carregamento e adição, há variações desta instrução dependendo do modo de endereço.

LP de Baixo Nível e Pseudocódigo

- **Pep/8: exemplos de opcodes de instruções**

– **01001**: *Entrada de caracteres no operando*: Insira um caractere ASCII da entrada padrão. Somente o endereçamento direto é permitido, portanto, o caractere é armazenado no endereço mostrado no especificador do operando.

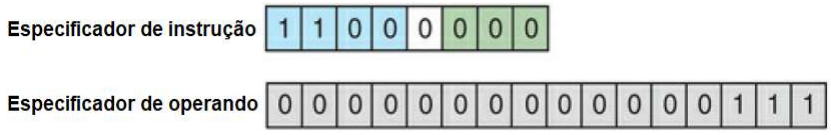
– **01010**: *Saída de caracteres do operando*: Envia um caractere ASCII para a saída padrão. O modo de endereçamento pode ser imediato ou direto.

15

LP de Baixo Nível e Pseudocódigo

- **Pep/8: exemplos de instruções**

– Instrução 1:



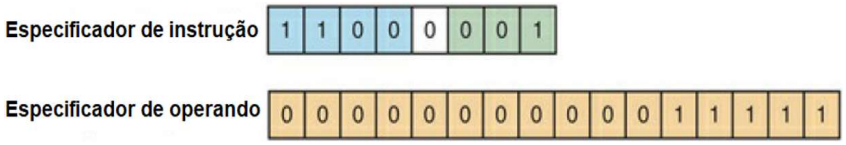
- O modo de endereçamento é imediato, ou seja, o valor a ser carregado no registrador A está no especificador de operando (cor cinza).
- Após a execução desta instrução, o conteúdo do especificador de operando (que é dado) seria carregado no acumulador (registrador A), i.e., o registrador A conteria 0007 e o conteúdo original (anterior) do registrador A seria perdido.

16

LP de Baixo Nível e Pseudocódigo

- **Pep/8: exemplos de instruções**

– Instrução 2:



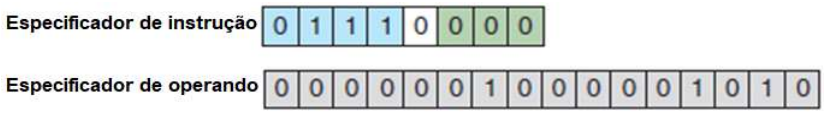
- O modo de endereçamento é direto. O operando (dado) não está no especificador de operando; neste consta o endereço 001F (laranja) do operando na memória.
- O conteúdo do endereço 001F será carregado no acumulador (registrador A). O registrador A tem uma palavra de 2 bytes, então o conteúdo das posições 001F e 0020 é carregado no registrador A.

17

LP de Baixo Nível e Pseudocódigo

- **Pep/8: exemplos de instruções**

– Instrução 3:



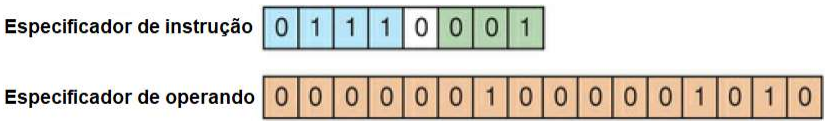
- Assim como a operação de carregamento, a operação de adição usa o especificador do modo de endereçamento, oferecendo interpretações alternativas.
- O conteúdo do especificador do operando (valor 522_{10}) é somado ao conteúdo do registrador A.
  – O especificador de operando está sombreado (cinza) para mostrar que se trata de dados.

18

LP de Baixo Nível e Pseudocódigo

- **Pep/8: exemplos de instruções**

– Instrução 4:



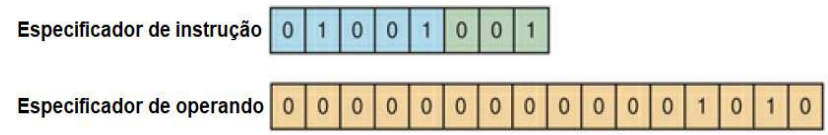
- Mesma instrução do slide anterior, diferente apenas no modo de endereçamento (aqui 001).
- Como o especificador do modo de endereço é direto (001), o conteúdo do especificador de operando é o endereço (020A) do dado a ser buscado na memória e somado ao registrador A.

19

LP de Baixo Nível e Pseudocódigo

- **Pep/8: exemplos de instruções**

– Instrução 5:



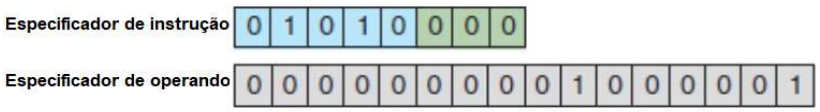
- Essa operação só permite endereçamento direto.
- Insere um caractere ASCII lido do dispositivo de entrada no endereço indicado pelo especificador de operando, ou seja, no endereço de memória 000A.

20

LP de Baixo Nível e Pseudocódigo

- **Pep/8: exemplos de instruções**

– Instrução 6:



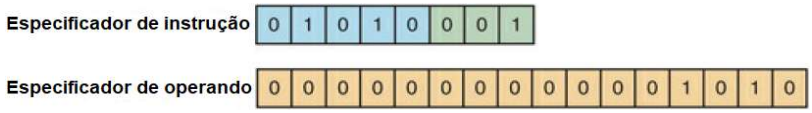
- Envia o caractere ASCII do especificador de operando, para o dispositivo de saída. O modo de endereçamento pode ser imediato ou direto.
- O especificador de operando contém 1000001, que é 41_{16} ou 65_{10}, ou seja, o caractere é 'A' é escrito na saída, i.e., na tela.

21

LP de Baixo Nível e Pseudocódigo

- **Pep/8: exemplos de instruções**

– Instrução 7:



- Mesma instrução do slide anterior, diferente apenas no modo de endereçamento (aqui 001).
- Como o endereçamento é direto, escreve o caractere ASCII armazenado no endereço (000A) indicado no especificador de operando.
- Não sabemos o que tem no endereço 000A. O caractere ASCII armazenado nesse endereço é impresso.

22

LP de Baixo Nível e Pseudocódigo

- **Um algoritmo “Hello”**

Ação	Instrução Binária	Instrução em Hexadecimal
Write 'H'	01010000 0000000001001000	50 0048
Write 'e'	01010000 0000000001100101	50 0065
Write 'l'	01010000 0000000001101100	50 006C
Write 'l'	01010000 0000000001101100	50 006C
Write 'o'	01010000 0000000001101111	50 006F
Stop	00000000	00

– O programa acima tem seis instruções: cinco para escrever um caractere e uma para indicar o fim do processo.
– A instrução para escrever um caractere na tela é 0101, a operação "Saída de caractere do operando".

23

LP de Baixo Nível e Pseudocódigo

- **Uma simulação manual**

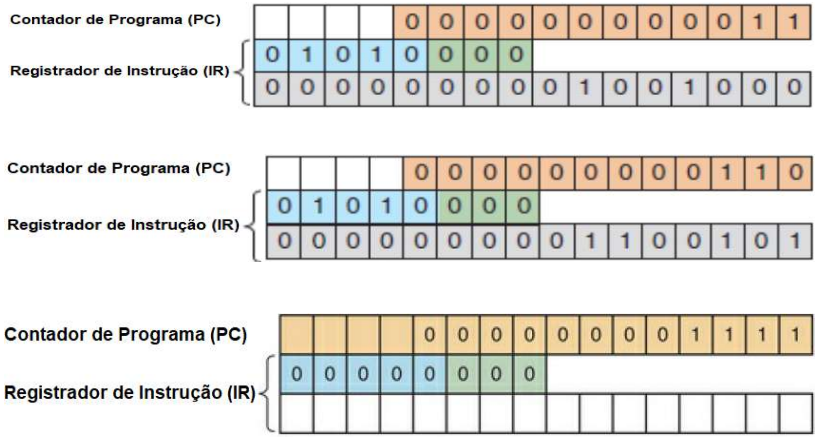
– Lembrando: seis instruções em nosso programa exemplo.
– Supomos que elas estão em posições contíguas na memória, com a primeira instrução armazenada nas posições de memória 0000 – 0002.
– A execução começa carregando 0000 no contador de programa (PC).
– Em cada etapa da execução, vamos examinar o PC (em laranja) e o registrador de instruções (IR).
– O programa não acessa o registrador A, portanto, não nos preocupamos em mostrá-lo.
– Ao final da primeira busca, o PC e o IR se parecem com o diagrama a seguir. Observe que o contador de programa é incrementado assim que a instrução é acessada.

24

LP de Baixo Nível e Pseudocódigo

- **Uma simulação manual**

– Na execução das duas primeiras e da ultima instrução:



25

25

LP de Baixo Nível e Pseudocódigo

- **Simulador Pep/8**

– **Simulador Pep/8** é um programa que se comporta exatamente como a máquina virtual Pep/8.

– Para executar um programa, digite o código hexadecimal, byte a byte, com espaços entre cada byte.

  - O “zz” indica fim do programa.



26

26

LP de Baixo Nível e Pseudocódigo

- **Simulador Pep/8**

– O que o programa abaixo faz?

Ação	Instrução Binária	Instrução em Hexadecimal
Entre uma letra na localização F	01001001 0000000000001111	49 000F
Entre uma letra em F + 1	01001001 0000000000001000	49 0010
Escreva a segunda letra	01010001 0000000000001000	51 0010
Escreva a primeira letra	01010001 0000000000001111	51 000F
Stop	00000000	00

27

LP de Baixo Nível e Pseudocódigo

- **Linguagem Assembly**

– **Linguagem Assembly** é uma linguagem que usa códigos mnemônicos para representar instruções em linguagem de máquina.

– **Assembler** é um programa que lê cada uma das instruções em formato mnemônico e as traduz para o equivalente em linguagem de máquina.

28

LP de Baixo Nível e Pseudocódigo

- **Linguagem Assembly do Pep/8**

Mnemônico	Operando, Especificador de Modo	Significado da Instrução
STOP		Interrompa a execução.
LDA LDA	0x008B, i 0x008B, d	Carregue 008B no registrador A. Carregue o conteúdo da posição 8B no registrador A.
STA	0x008B, d	Armazene o conteúdo do registrador A na posição 8B.
ADDA ADDA	0x008B, i 0x008B, d	Adicione 008B ao registrador A. Adicione o conteúdo da posição 8B ao registrador A.
SUBA SUBA	0x008B, i 0x008B, d	Subtraia 008B do registrador A. Subtraia o conteúdo da posição 8B do registrador A.
BR		Desvie para a posição especificada no especificador de operando.
CHARI	0x008B, d	Lê um caractere e armazena-o na posição 8B.
CHARO	0x008B, i 0x000B, d	Escreva o caractere 8B. Escreva o caractere armazenado na posição 0B.
DECI DEC0 DEC0	0x008B, d 0x008B, i 0x008B, d	Lê um número decimal e armazena-o na posição 8B. Escreva o número decimal 139 (8B em hexadecimal). Escreva o número decimal armazenado na posição 8B.

– Lembra-se da diferença entre endereçamento imediato e direto?
  - i: imediato
  - d: direto

LP de Baixo Nível e Pseudocódigo

- **Linguagem Assembly do Pep/8**

Pseudo OP	Argumento	Significado
.ASCII	"Str\x00"	Representa uma sequência de bytes ASCII.
.BLOCK	Número de bytes	Cria um bloco de bytes.
.WORD	Valor	Cria uma palavra e armazena um valor nela.
.END		Sinaliza o fim do programa em linguagem assembly.

– Qual é a diferença entre operações e pseudo-operações?

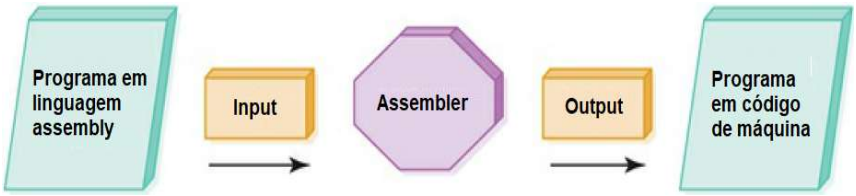
Assembler Listing

Addr	Code	Mnemon	Operand	Comment
0000	500048	CHARO	0x0048, i	
0003	500065	CHARO	0x0065, i	
0006	50006C	CHARO	0x006C, i	
0009	50006C	CHARO	0x006C, i	
000C	50006F	CHARO	0x006F, i	
000F	00	STOP		

LP de Baixo Nível e Pseudocódigo

- **Processo de montagem**

– O processo de execução de um programa codificado em linguagem assembly.



– Entrada para o montador: programa em linguagem assembly.
– Saída do montador: programa em código de máquina.
– A criação da linguagem assembly removeu muitos detalhes da programação em linguagem de máquina, abstraindo as instruções em palavras (mneumônicos).
– Esta etapa facilitou a vida do programador.

31

LP de Baixo Nível e Pseudocódigo

- **Um novo programa**

– Problema: Leia e some três valores e imprima a soma, em assembly do MIPS 32.

```
li $v0, 5              # Código para ler inteiro
syscall                # Lê o primeiro número
move $s0, $v0          # Armazena o primeiro número em $s0

li $v0, 5              # Código para ler inteiro
syscall                # Lê o segundo número
move $s1, $v0          # Armazena o segundo número em $s1

li $v0, 5              # Código para ler inteiro
syscall                # Lê o terceiro número
move $s2, $v0          # Armazena o terceiro número em $s2

add $t0, $s0, $s1      # Soma o primeiro com o segundo número
add $s3, $t0, $s2      # Adiciona o terceiro número ao resultado

li $v0, 1
move $a0, $s3          # Move o resultado para $a0
syscall                # Imprime o resultado da soma
```

32

LP de Baixo Nível e Pseudocódigo

- **Um novo programa**

– Problema: Leia e some três valores e imprima a soma, em linguagem de máquina (em binário) do MIPS 32.

```
00110100 00000010 00000000 00000101     // li $v0, 5
00000000 00000000 00000000 00001100     // syscall
00000000 01000000 10000000 00100001     // move $s0, $v0
00110100 00000010 00000000 00000101     // li $v0, 5
00000000 00000000 00000000 00001100     // syscall
00000000 01000000 10001000 00100001     // move $s1, $v0
00110100 00000010 00000000 00000101     // li $v0, 5
00000000 00000000 00000000 00001100     // syscall
00000000 01000000 10010000 00100001     // move $s2, $v0
00000010 00010001 01000000 00100000     // add $t0, $s0, $s1
00000001 00010010 10011000 00100000     // add $s3, $t0, $s2
00110100 00000010 00000000 00000001     // li $v0, 1
00000010 01100000 00100000 00100001     // move $a0, $s3
00000000 00000000 00000000 00001100     // syscall
```


LP de Baixo Nível e Pseudocódigo

- **Pseudocódigo**

– Problema: Leia e some os valores limite.

- Quantos valores?
- De onde vem esse valor?
  - Será necessário repetir a leitura e a soma.
- Como saberemos quando terminarmos?

– O que é **pseudocódigo**?

- Uma combinação de linguagem natural (inglês ou português) e formatação para explicitar as etapas de um algoritmo.

LP de Baixo Nível e Pseudocódigo

- **Um exemplo de pseudocódigo**

– Algoritmo para converter um número da base 10 para uma outra base:

```
Enquanto o quociente for diferente de 0
   Divida o número decimal pela nova base
   Faça o resto ser o próximo dígito à esquerda na resposta
   Substitua o número decimal original pelo quociente
```

35

Referencias Bibliográficas

– Nell Dale; John A. Lewis. Computer Science Illuminated. 6ed. Jones and Bartlett, 2016.

– Behrouz Forouzan e Firouz Mosharraf. Fundamentos da Ciência da Computação. 2ed. Cengage Learning, 2012.

– J. Gleen Brookshear. Ciência da Computação: uma visão abrangente. 11ed. Bookman, 2013.

– Marcelo Marcula; Pio Armando Benini Filho. Informática: conceitos e aplicações. 4ed. Érica, 2013.

– Ricardo Daniel Fedeli; Enrico Giulio Franco Polloni; Fernando Eduardo Peres. Introdução à ciência da computação. 2ed. Cengage Learning, 2010.

36


DÚVIDAS


QUEM NÃO ENTENDEU, LEVANTE A MÃO
O QUE VOCÊ NÃO ENTENDEU, MANOLITO?
NADA, DESDE MARÇO ATÉ AGORA

37

37